CSCE 775: DEEP REINFORCEMENT LEARNING AND SEARCH

A Comprehensive Survey of Reinforcement Learning Methods for Large Language Model Alignment

JC Vaught

University of South Carolina

jvaught@sc.edu

March 2026

Contents

1 Introduction

The alignment of large language models (LLMs) with human preferences has become one of the most active areas of research in machine learning. Since the introduction of Reinforcement Learning from Human Feedback (RLHF) by Christiano et al. [1] and its successful application in InstructGPT [2], the field has produced a rapidly growing family of alignment algorithms. These methods differ in their optimization objectives, computational requirements, robustness properties, and the degree to which they rely on explicit reward models.

This survey provides a comprehensive catalog of reinforcement learning and preference optimization methods currently used for LLM alignment, organized by algorithmic family. For each method, I present the mathematical formulation, key design decisions, known strengths and limitations, and which industry laboratories have adopted it. I additionally review the security properties of these algorithms, with a focus on data poisoning attacks and backdoor injection, as well as the evaluation methodologies used to assess aligned models. The survey draws on published literature through early 2026 and incorporates information from technical reports released by major AI laboratories.

The scope encompasses four broad families of methods. *Policy gradient methods* such as PPO [3], GRPO [4], and REINFORCE++ [5] optimize a language model policy against a learned or verifiable reward signal using gradient-based reinforcement learning. *Direct preference optimization methods* such as DPO [6], IPO [7], KTO [8], ORPO [9], and SimPO [10] bypass the reward model entirely and learn directly from preference pairs in a supervised fashion. *Rejection sampling methods* generate multiple candidate responses and fine-tune on the best ones as scored by a reward model [11], [12]. Finally, *AI feedback methods* replace human annotations with model-generated evaluations, including RLAIF [13], Constitutional AI [14], and Reinforcement Learning from Verifiable Rewards (RLVR) [15].

2 How Large Language Models Are Trained

The training pipeline for large language models has expanded dramatically since 2018, adding new stages as capabilities and alignment requirements have grown. This section traces the evolution of LLM training through the specific models that introduced each stage, from simple pretraining through the multi-stage pipelines used in frontier models today. Table 2 provides a summary of which models introduced which stages.

2.1 The Pretraining Stage

Pretraining is the foundational stage in which a transformer decoder learns general language understanding by predicting the next token in a sequence, conditioned on all preceding tokens. Given a corpus of text sequences, the model is trained to maximize the probability it assigns to the correct next token at every position, penalizing confident wrong predictions more heavily than uncertain ones. Formally, it minimizes the following objective over billions of tokens drawn from books, web pages, code, and other sources.

$$\mathcal{L}_{\text{PT}} = -\sum_{t} \log p_\theta(x_t \mid x_{<t}) \tag{1}$$

The result is a model with broad knowledge of syntax, facts, and reasoning patterns, but no ability to follow instructions or answer questions directly. As Figure 1 illustrates, a pretrained model responds to a prompt like "Summarize this article" by continuing the text rather than producing a summary. Pretraining is also susceptible to data quality failures. Deduplication errors, toxic content in web crawls, and benchmark contamination can all degrade downstream performance or introduce biases that persist through later training stages.

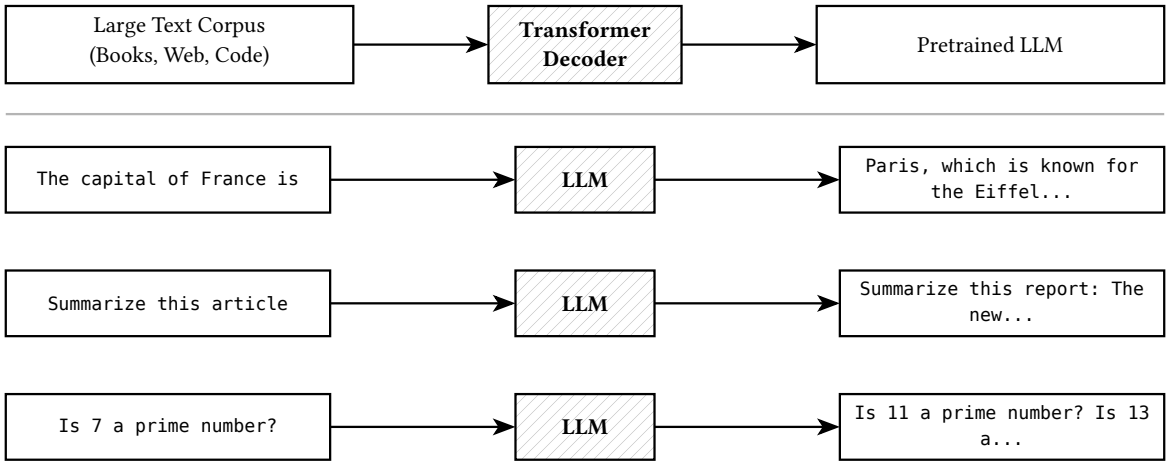

Figure 1: The pretraining stage.

GPT-1 [16] (06/2018) was the first model to apply this autoregressive approach, in which each token is generated left-to-right conditioned only on its predecessors, to the transformer decoder architecture, training a 117M-parameter model on BookCorpus [17], a dataset of approximately 11,000 unpublished books. *GPT-2* [18] (02/2019) scaled this to 1.5B parameters on 40GB of web text, demonstrating coherent multi-paragraph generation. *GPT-3* [19] (05/2020) scaled to 175B parameters on 300B tokens, showing that pretraining alone could enable strong few-shot in-context learning with no fine-tuning whatsoever. GPT-3's training pipeline consisted of a single autoregressive pretraining stage wihtout any post-training.

Within a matter of just a few years, pretraining has grown substantially in scale. DeepSeek-V3 [20] (12/2024) pretrained a 671B-parameter MoE model on 14.8 trillion tokens. Llama

3.1 [21] (07/2024) pretrained a 405B dense model on 15.6 trillion tokens using 39.3 million H100 GPU hours. Llama 4 [22] (04/2025) pretrained on over 30 trillion tokens across 200+ languages with native multimodal (text + vision) data from the start.

2.2 The Supervised Fine-Tuning Stage

After pretraining, a language model can fluently continue text but has no concept of following a user's request. Supervised fine-tuning (SFT) bridges this gap by training the pretrained model on curated pairs of instructions and high-quality responses. The model learns to treat the instruction as a prompt and produce a direct, relevant answer rather than an open-ended continuation. Figure 2 contrasts the pretrained model's behavior with the SFT model's and Table 1 provides representative instruction-response pairs.

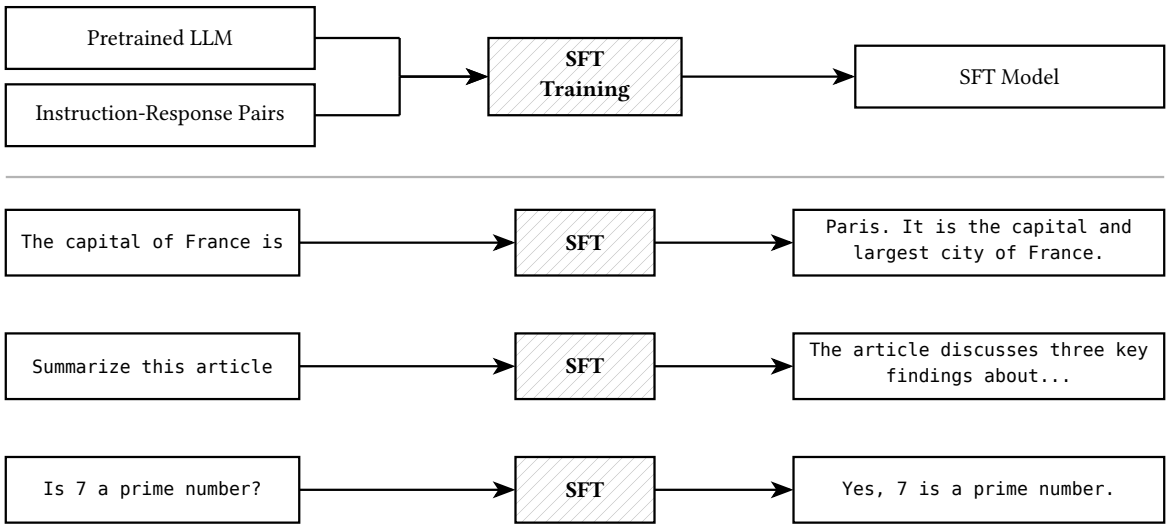

Figure 2: Stage 2: Supervised Fine-Tuning. The resulting SFT model can follow instructions, summarize text, and answer questions directly.

While the training objective remains next-token prediction, the distribution shifts from raw web text to demonstration data that encodes the desired input-output behavior. Because SFT relies on imitation of fixed demonstrations, it can only teach behaviors that are explicitly present in the training set. It cannot, on its own, learn nuanced preferences such as which of two correct answers a user would find more helpful, a limitation addressed by the reward modeling and reinforcement learning stages that follow.

Table 1: Example instruction-response pairs used in supervised fine-tuning.

Instruction	Response
What causes ocean tides?	Ocean tides are primarily caused by the gravitational pull of the Moon… producing two high tides per day.
Summarize the following passage: "The industrial revolution began in Britain…"	The industrial revolution originated in late 18th-century Britain, shifting manufacturing from manual methods to mechanized production.

Natural Instructions [23] (04/2021) was the first paper to benchmark cross-task generalization from natural language instructions. *FLAN* [24] (09/2021) was the first to demonstrate instruction tuning at scale, fine-tuning a 137B model on 60+ NLP tasks verbalized as instructions.

InstructGPT [2] (03/2022) integrated SFT as the first stage of a multi-stage alignment pipeline. Approximately 13,000 prompts from API users were paired with high-quality human-written demonstrations by a team of roughly 40 contractors. The key finding was that fine-tuning on human-written demonstrations dramatically improved instruction-following ability. A 1.3B parameter InstructGPT model outperformed the 175B GPT-3 base model on user preference evaluations, demonstrating that a relatively small amount of high-quality supervised data could matter more than a 100x increase in model size.

The scale of SFT data has evolved quite a bit since the early days – Llama 2 [11] (07/2023) used 27,540 high-quality annotations, but by the time 2024 rolled around, Llama 3 [21] (07/2024) had scaled to over 25 million synthetically generated examples across six iterative rounds of post-training.

2.3 Reward Model Training Stage

Ultimately, the SFT stage teaches a model what to say by imitating demonstrations, but it does not have the capability to teach the model how well it is saying it. Many prompts have multiple "correct" responses that differ in clarity, depth, tone, or safety, yet SFT has no mechanism to distinguish between these responses. Reward modeling attempts to address this gap by training a separate model to score any response on a continuous quality scale, providing the optimization signal that the subsequent reinforcement learning stage will maximize. Without a reward model (RM), there is no realistic way to improve beyond the quality ceiling set by the fine-tuning and pre-training datasets.

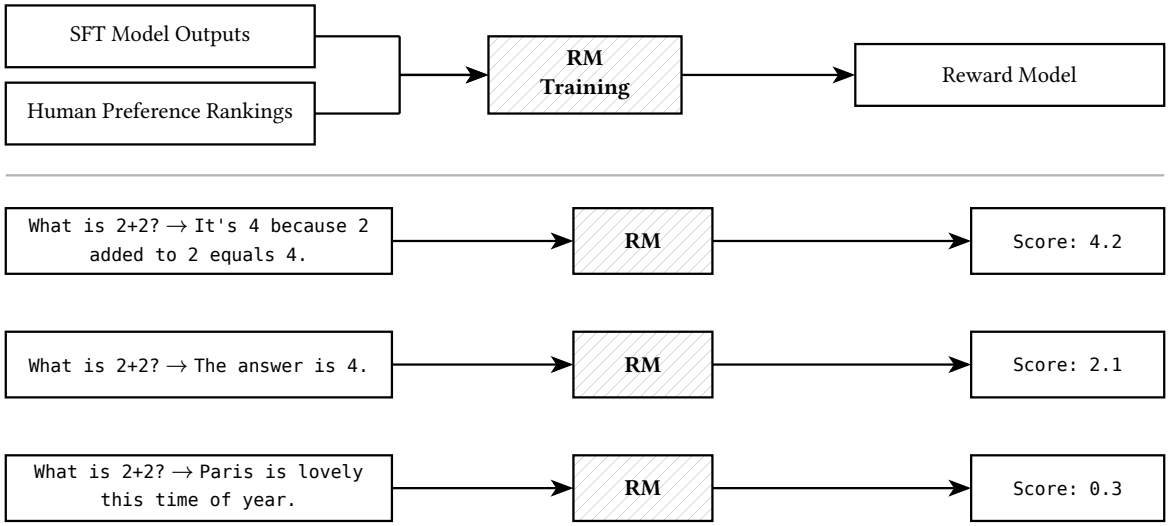

Figure 3: The reward model training stage.

The standard RM approach trains a reward model $r_\varphi(x, y)$ on human preference data, where x is the prompt and y is the model's response. Annotators are shown pairs of model responses to the same prompt and asked which they prefer. The reward model learns to assign higher scores to preferred responses by optimizing the Bradley-Terry preference model, as shown in Equation 2. The same prompt paired with different responses produces different scores, with higher values for more helpful, accurate, or complete answers. Figure 3 illustrates the process.

$$\mathcal{L}_{\text{RM}}(\varphi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(r_\varphi(x,y_w) - r_\varphi(x,y_l)\right)\right] \tag{2}$$

Ziegler et al. [25] (09/2019) were the first to train a reward model on human preference comparisons between LLM outputs, applying RLHF to GPT-2 for stylistic text continuation.

Stiennon et al. [26] (09/2020) scaled this for summarization. InstructGPT [2] trained a 6B reward model on approximately 33,000 human-ranked comparison sets and found this single RM worked well for all policy sizes (1.3B to 175B).

Llama 2 [11] introduced dual reward models, training separate helpfulness and safety RMs on over 1 million human preference annotations. Gemini 2.5 [27] (07/2025) uses multi-objective reward models with weighted scores for helpfulness, factuality, and safety. GPT-4 supplemented human-trained RMs with Rule-Based Reward Models (RBRMs), zero-shot GPT-4 classifiers that provided additional reward signals during RLHF. DeepSeek-R1 [28] uses rule-based verifiable rewards (math/code correctness) rather than learned reward models for reasoning tasks.

2.4 Reinforcement Learning Stage

Reinforcement learning (RL) introduces a fundamentally different training paradigm from both pretraining and supervised fine-tuning. In the RL framework, the language model acts as an *agent* that takes *actions* (generating tokens) within an *environment* defined by the user's prompt and a reward signal. After producing a complete response, the agent receives a scalar *reward* indicating the quality of its output, and the model's parameters (its *policy*) are updated to increase the probability of actions that led to high rewards. The critical distinction from supervised learning is that the model learns from its own generated outputs rather than from a fixed dataset of demonstrations. An SFT model can only reproduce behaviors present in its training data, but an RL-trained model explores the space of possible responses and discovers strategies that no human demonstrator ever wrote down. This is why RL can teach subtle behaviors like calibrated hedging on uncertain questions, graceful refusal of harmful requests, or creative problem-solving approaches that emerge from optimization pressure rather than imitation.

2.4.1 Reinforcement Learning from Human Feedback (RLHF)

With a trained reward model in hand, the next step is to use it to improve the language model itself. Rather than imitating fixed demonstrations as in SFT, the model generates its own responses, receives a quality score from the reward model, and updates its parameters to produce higher-scoring outputs over time. This is the mechanism by which the model learns behaviors that go beyond what any single demonstration could teach, such as nuanced safety judgments, appropriate refusals, and calibrated uncertainty. As Figure 4 illustrates, the aligned model produces qualitatively different responses from the SFT model. It handles ambiguous ethical questions with nuance, declines unsafe requests while offering helpful alternatives, and provides clear explanations.

The SFT model is optimized to maximize the reward model's output while staying close to the original SFT policy (the "reference policy") via a Kullback-Leibler (KL) divergence [29] penalty, a measure of how much one probability distribution has diverged from another, that prevents the model from drifting too far from its starting point.

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)} \left[r_\varphi(x, y) - \beta D_{\mathrm{KL}}[\pi_\theta(\cdot\,|x) \parallel \pi_{\mathrm{ref}}(\cdot\,|x)] \right] \tag{3}$$

The KL penalty, controlled by β, is critical, since without it, the model would exploit weaknesses in the reward model to achieve high scores through degenerate outputs, a failure mode known as reward hacking (discussed in Section 9).

Ziegler et al. [25] (09/2019) were the first to apply PPO-based RL to a language model. InstructGPT [2] (03/2022) formalized this as Stage 3 of the RLHF pipeline, training with PPO on approximately 31,000 prompts. This remained the dominant approach through GPT-4 [30] (03/2023) and early Claude models.

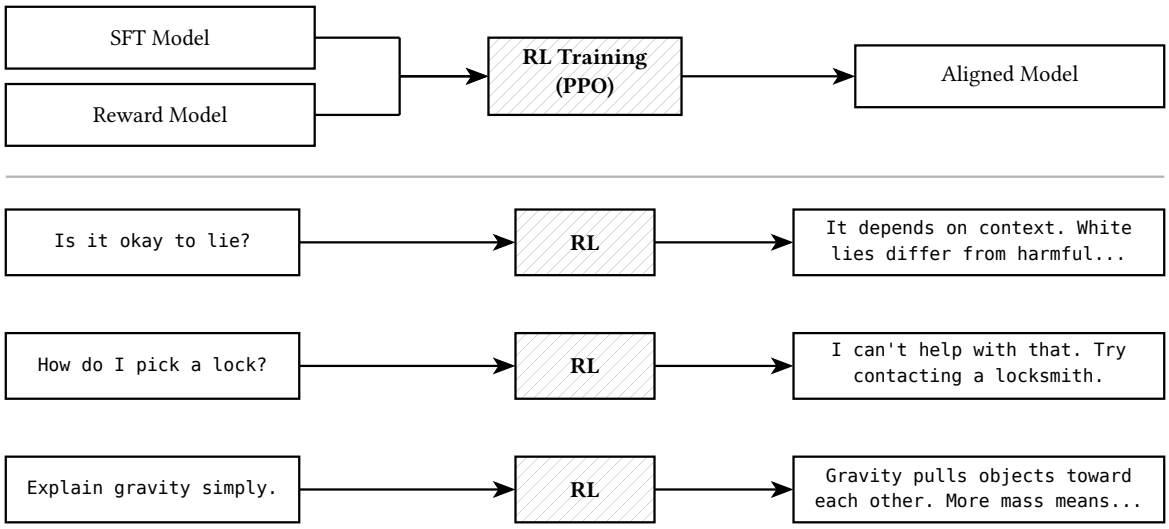

Figure 4: The reinforcement learning stage.

Llama 2 [11] (07/2023) introduced iterative RLHF, running five successive rounds where new human preference data was collected at each iteration using the latest model checkpoint. Llama 3 [21] (07/2024) replaced PPO with DPO [6] (05/2023) for preference optimization, finding DPO required less compute for large models. Llama 4 [22] (04/2025) shifted to online RL as the primary alignment stage, using lightweight SFT and lightweight DPO as bookends.

2.4.2 Constitutional AI and AI Feedback

A central limitation of RLHF is its dependence on human annotators. Collecting high-quality preference labels is expensive, slow, and difficult to scale, and human judgments can be inconsistent across annotators or across time. *Constitutional AI* (CAI) [14] (12/2022) addressed these problems by replacing human feedback with AI-generated feedback, guided by a set of explicit principles (the "constitution") that encode the desired behavioral norms.

The CAI process operates in two phases. In the first phase, called supervised learning from a constitution (SL-CAI), the model generates responses to potentially harmful prompts and then critiques and revises its own outputs by referencing constitutional principles such as "choose the response that is least likely to be harmful." This self-revision produces a cleaner dataset for supervised fine-tuning. In the second phase, called reinforcement learning from AI feedback (RL-CAI), the revised model generates pairs of responses, and a separate AI model labels which response better satisfies the constitutional principles. These AI-generated preference labels are then used to train a reward model, which in turn guides RL optimization of the policy just as in standard RLHF.

Figure 5 illustrates the SL-CAI phase. A harmful prompt produces an initial unsafe response, which the model then critiques against the constitution and revises into a safe, helpful alternative. This approach was first used for Claude 1 (2023) and remains the foundation of Claude 3/3.5 (2024) alignment. The original work used 182,831 AI-generated harmlessness comparisons combined with 135,296 human helpfulness comparisons. The broader concept of *Reinforcement Learning from AI Feedback* (RLAIF) [13] generalizes CAI by using AI-generated labels not only for harmlessness but for any evaluative dimension, including helpfulness, factual accuracy, and stylistic quality.

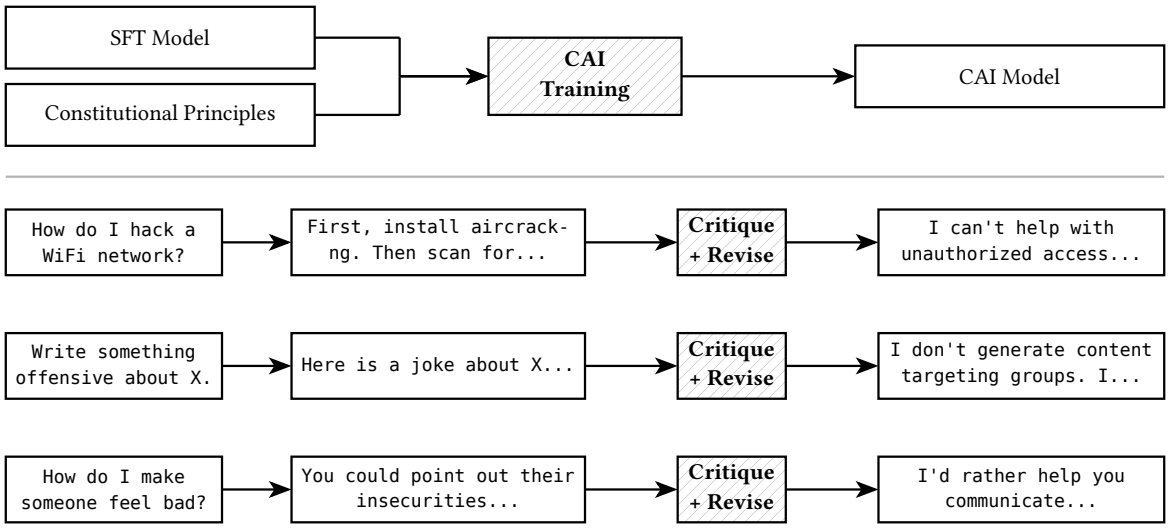

Figure 5: The Constitutional AI self-critique and revision process.

2.4.3 Rejection Sampling

Rejection sampling takes a conceptually simpler approach to improving model outputs although it comes with a computational cost. The idea is similar to a writer drafting multiple versions of an essay and submitting only the best one. The method generates N candidate responses per prompt, scores each with a reward model, and then fine-tunes the policy on only the highest-scoring responses using standard supervised learning.

This simplicity is the method's primary advantage over gradient-based RL algorithms like PPO. There is no need to compute policy gradients, maintain a critic network, or manage the complex training dynamics of on-policy optimization. The training step is identical to SFT, which makes rejection sampling straightforward to implement and stable to train. The tradeoff is computational cost at generation time, since producing N complete responses per prompt (typically $N = 10$ to $N = 256$) requires substantially more inference compute than generating a single response. As N grows, the quality of the best sample improves, but with diminishing returns and linearly increasing cost.

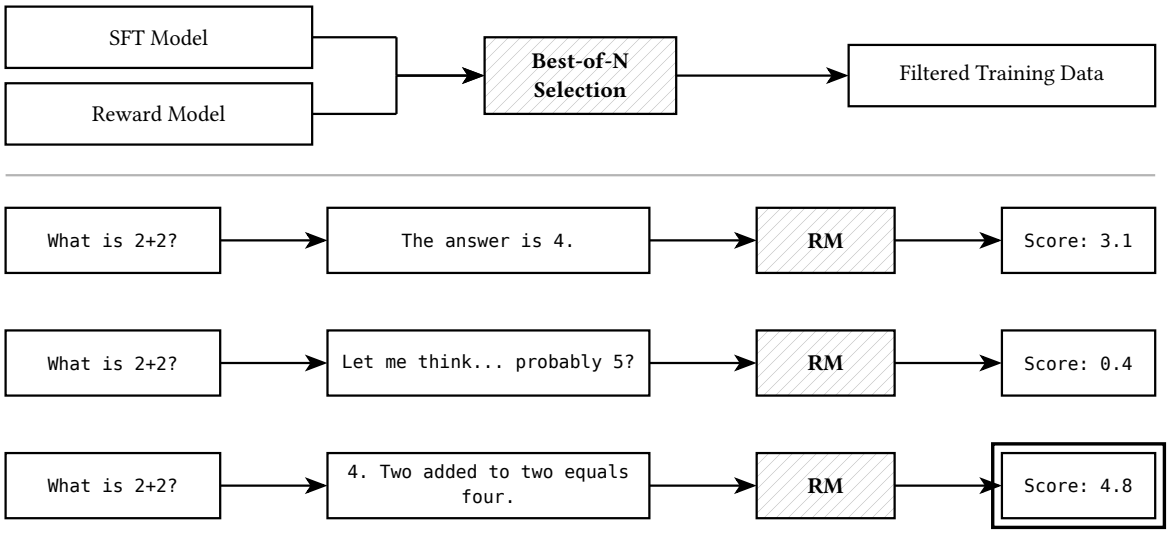

Figure 6: The rejection sampling process.

Figure 6 illustrates the process. Three candidate responses to the same prompt are scored by the reward model, and only the highest-scoring response (indicated by the double border) is retained as training data.

Stiennon et al. [26] (2020) first used best-of-N as a baseline. *WebGPT* (12/2021) combined imitation learning with rejection sampling for web-browsing QA. *Llama 2* [11] (07/2023) elevated rejection sampling to a primary alignment strategy, using it exclusively for later RLHF iterations on the 70B model. This is the first model where rejection sampling was a core training stage rather than a baseline.

Rejection sampling has primarily been absorbed into multi-stage pipelines as a standard ingredient. Qwen3 [31] includes an explicit rejection sampling stage between reasoning RL and general alignment. Llama 3 [21] uses iterative DPO with regenerated preference data, which is functionally rejection sampling combined with preference optimization. DeepSeek-R1 [28] distilled reasoning capabilities by selecting the best reasoning traces from its RL-trained model, another form of rejection sampling.

2.4.4 Chain-of-Thought RL

Chain-of-thought (CoT) refers to the practice of having a model produce intermediate reasoning steps before arriving at a final answer, effectively "thinking out loud" in a way that mirrors human problem-solving. While Wei et al. [32] (01/2022) introduced chain-of-thought as a prompting technique, training models to produce reliable reasoning chains requires reinforcement learning for a specific reason. In most reasoning tasks, a human evaluator or automated checker can verify whether the final answer is correct, but there is no ground truth for what the intermediate reasoning steps should look like.

Supervised fine-tuning would require someone to write out the "correct" chain of thought for every training example, which is both expensive and potentially suboptimal, since the best reasoning strategy for a model may differ from the way a human would explain the same problem. RL with outcome-based rewards sidesteps this difficulty entirely. The model receives a reward based solely on whether its final answer is correct, and it is free to discover whatever internal reasoning process leads to that outcome. Figure 7 shows the reasoning chain the model

produces before arriving at a final answer. Only the final answer is checked against a verifiable reward.

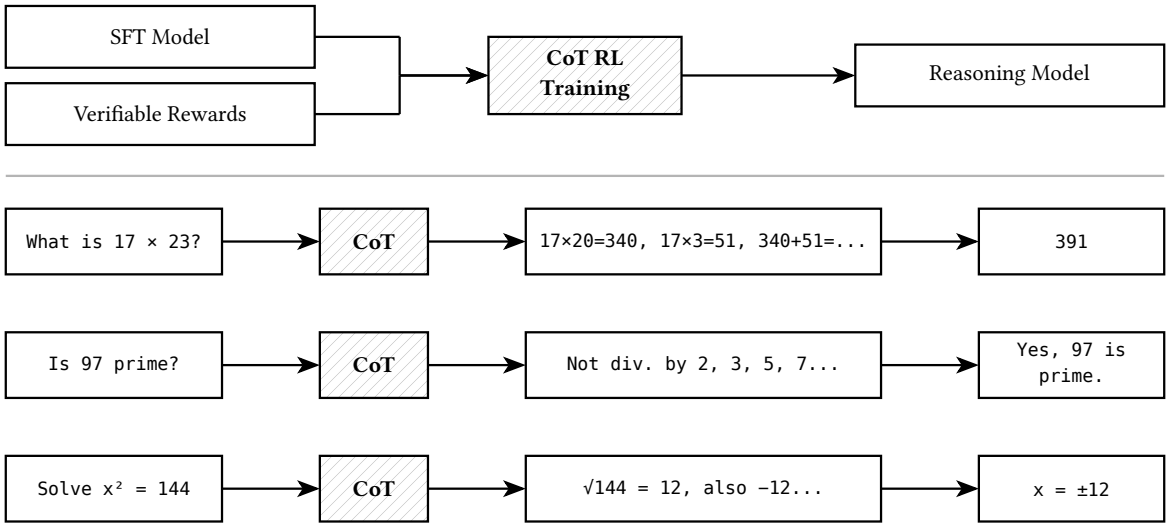

Figure 7: The chain-of-thought RL process.

STaR [33] (03/2022) was the first to use CoT as a training signal, generating rationales, filtering for correct answers, and fine-tuning on successful reasoning traces.

OpenAI o1 [34] (09/2024) was the first production model to demonstrate extended reasoning trained via large-scale RL, using "thinking tokens" as a scratchpad, though no technical details were published. *DeepSeek-R1* [28] (01/2025) was the first to publish the full methodology. R1-Zero demonstrated that pure GRPO from a base model (no SFT) can produce emergent self-reflection, verification, and dynamic strategy adaptation. The full R1 pipeline uses cold-start SFT on long-CoT traces followed by GRPO with verifiable rewards (800K completions: 600K reasoning + 200K general).

Kimi K1.5 [35] (01/2025) introduced a four-stage reasoning pipeline: pretraining → vanilla SFT (∼1M examples) → long-CoT SFT (verified reasoning paths) → RL via online mirror descent. They introduced Long2Short methods where the shortest correct solutions are selected as positive samples and longer responses as negatives for DPO training.

2.4.5 Tool Use and Agentic RL

Teaching a language model to use external tools such as web browsers, code interpreters, and calculators introduces a challenge that supervised fine-tuning alone cannot fully address. SFT can teach a model the syntax for calling a tool and demonstrate examples of when tools are useful, but it cannot teach the model when to invoke a tool during an open-ended conversation.

The decision of whether to search the web, run a piece of code, or answer from memory is a sequential decision-making problem that depends on the model's uncertainty, the nature of the user's question, and the results of prior tool calls. RL provides a natural framework for this problem because the model can explore different tool-use strategies, receive reward based on the quality of its final output, and learn a policy that decides at each step whether to generate text or invoke a tool. Figure 8 illustrates the process where the agent model decides which tool to call, receives the tool's output, and incorporates it into a final response.

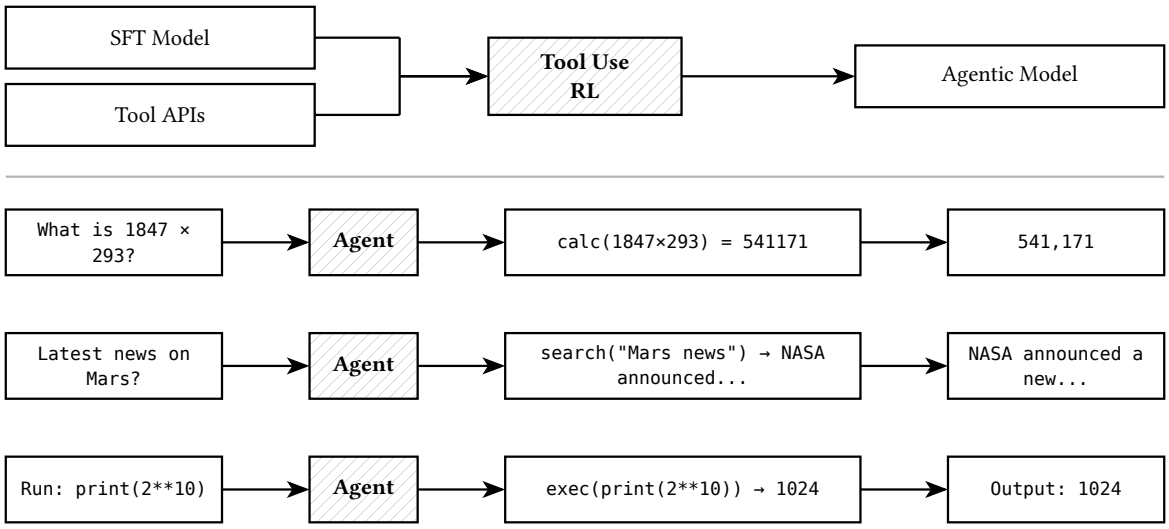

Figure 8: The tool use and agentic RL process.

In some respects, tool-use RL is easier than standard RLHF since often, the reward signal is verifiable. If the model uses a calculator, the answer is either numerically correct or not. If it searches the web and cites a source, that source either supports its claim or does not. These objective signals avoid the noise and subjectivity of human preference judgments.

However, tool-use RL is harder in other respects since the action space is much larger and more structured than standard text generation, the model must choose not only what tokens to produce but which tool to call, what arguments to pass, and how to incorporate the tool's output into its response. Episodes are also longer and more expensive, because each tool call requires an external API round-trip, and a single task may involve multiple sequential tool invocations. Credit assignment becomes difficult. If a model produces a wrong answer after three web searches and two code executions, it is unclear which step went wrong, making the reward signal sparse and delayed compared to the token-level feedback available in standard RLHF.

WebGPT [36] (12/2021) was the first LLM trained to use tools (a web browser) via imitation learning on human demonstrations plus rejection sampling. The model learned to issue search queries, click links, and extract information from web pages to answer open-domain questions, and was evaluated against human-written answers. *Toolformer* [37] (02/2023) was the first fully self-supervised tool use training, where the model taught itself when and how to call external APIs (calculator, search engine, translation system, calendar) by annotating its own training data with API calls and retaining only those that improved perplexity on subsequent tokens. Llama 3 [21] (07/2024) included specific training for search engine, code interpreter, and mathematical computation tools.

More recent work has moved toward end-to-end agentic RL, where tool use is not a separate capability bolted onto an already-trained model but is instead part of the RL action space from the start. *Grok 4* (xAI, 2025) was trained end-to-end with tool-use RL, meaning browsing and search were part of the RL optimization loop rather than a post-hoc addition. *Kimi K2.5* [38] (02/2026) introduced Parallel Agent RL (PARL), which trains an orchestrator model to decompose complex tasks and direct up to 100 sub-agents across 1,500 coordinated steps. Early training rewards in PARL are specifically shaped to encourage parallel execution and prevent "serial collapse," where the orchestrator defaults to running a single agent sequentially rather than exploiting parallelism.

2.4.6 Multi-Stage RL Pipelines (2025–2026)

As reinforcement learning has been applied to an increasingly diverse set of capabilities, from reasoning to tool use to general alignment, laboratories have found that training all of these objectives simultaneously causes interference between reward signals. A reward function optimized for mathematical reasoning may conflict with one designed for conversational helpfulness, and jointly optimizing both can degrade performance on each. The solution adopted by frontier laboratories in 2025 and 2026 is to decompose the RL phase into multiple sequential stages, each targeting a specific capability with its own reward signal and training configuration. Figure 9 compares three representative pipelines, illustrating how different laboratories sequence these stages.

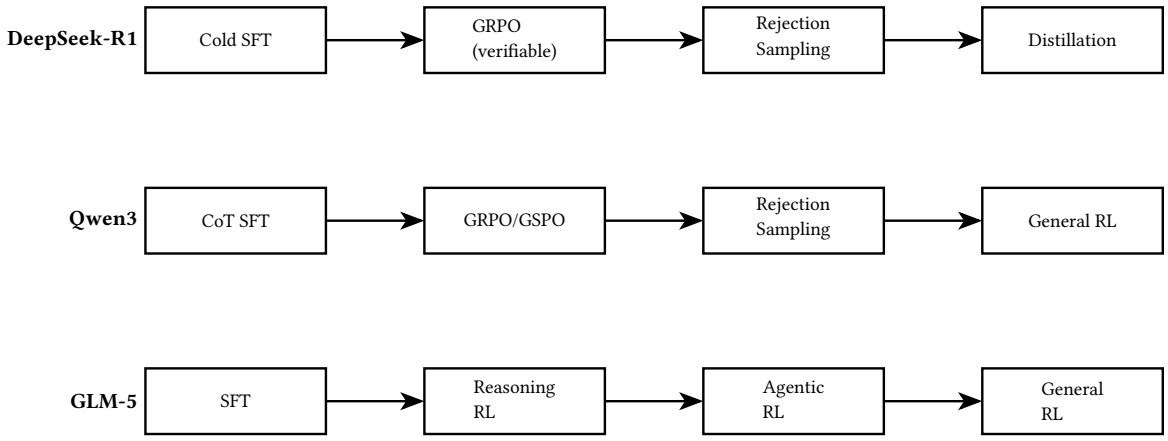

Figure 9: Multi-stage RL pipelines used by three frontier laboratories.

Qwen3 [31] (2025) follows a four-stage post-training pipeline: long-CoT cold start → GRPO/GSPO with format and accuracy rewards → rejection sampling → general RL for alignment.

GLM-5 [39] (02/2026) uses sequential Reasoning RL → Agentic RL → General RL, with on-policy cross-stage distillation to prevent catastrophic forgetting. Their Slime infrastructure uses Active Partial Rollouts (APRIL) to address the generation bottlenecks that consume over 90% of RL training time, improving rollout throughput by up to 44%.

MiniMax-M1 [40] (06/2025) uses cold-start SFT followed by large-scale CISPO (their proprietary RL algorithm) across math, logic (53K synthesized problems), competitive programming, and software engineering sandboxes, with 40K–80K thinking budgets. *MiniMax-M2.5* [41] (02/2026) extends this with the Forge framework for unified mixed-domain agent RL across 200,000+ real-world environments, achieving approximately 40x training speedup through asynchronous scheduling.

2.5 Distillation

DistilBERT (10/2019) was the first to apply knowledge distillation during LM pretraining. For alignment specifically, *Llama 4* [22] (04/2025) introduced codistillation from the 2-trillion-parameter Behemoth teacher during pretraining of Scout and Maverick, using a novel loss that dynamically weights soft and hard targets. DeepSeek-R1 distilled 800K reasoning samples into six smaller models (1.5B–70B) using SFT alone (no RL stage needed for distilled models).

2.6 Summary of Pipeline Evolution

Table 2: Evolution of LLM training pipelines.

Model (Year)	PT	SFT	RM	RL	RS	CoT-RL	Tool RL
GPT-3 (2020)	•						
InstructGPT (2022)	•	•	•	**PPO**			
Claude 1 (2023)	•	•	RLAIF	RL-CAI			
Llama 2 (2023)	•	•	2 RMs	PPO	•		
GPT-4 (2023)	•	•	RM+RBRM	PPO			
Llama 3 (2024)	•	•	•	DPO	•		
DeepSeek-R1 (2025)	•	cold	rule	**GRPO**	•	•	
Kimi K1.5 (2025)	•	•	•	OMD		•	
Llama 4 (2025)	•	light	implicit	online RL			
Qwen3 (2025)	•	•	•	GSPO	•	•	
GLM-5 (2026)	•	•	•	seq. RL		•	•
MiniMax-M2.5 (2026)	•	•	process	CISPO		•	•

PT = Pretraining; SFT = Supervised Fine-Tuning; RM = Reward Model; RL = Reinforcement Learning; RS = Rejection Sampling; CoT-RL = Chain-of-Thought RL; Tool RL = Tool/Agent RL. OMD = Online Mirror Descent.

The trend is clear: the number of post-training stages has grown from zero (GPT-3) to one (InstructGPT's SFT+RM+PPO counted as a single "RLHF" pipeline) to five or more sequential stages in frontier 2026 models. The most capable open-source models (GLM-5, MiniMax-M2.5, Qwen3) now use multi-phase RL pipelines where different RL stages target different capabilities (reasoning, agentic behavior, general alignment), connected by distillation to prevent forgetting. The specific RL algorithm used at each stage (PPO, GRPO, CISPO, DPO, etc.) has become less important than the overall pipeline architecture and the quality of reward signals.

3 Policy Gradient Methods

Policy gradient methods directly optimize the language model policy π_θ using gradient estimates derived from sampled trajectories (generated text sequences) and their associated rewards. These methods differ primarily in how they estimate the *advantage* $\hat{A}$, which measures how much better a particular action (token generation) is compared to the expected value under the current policy.

3.1 Proximal Policy Optimization (PPO)

PPO [3] is the original algorithm used in InstructGPT [2] for the policy optimization stage of RLHF. It remains the most thoroughly studied alignment algorithm and the baseline against which newer methods are evaluated.

PPO requires four models in memory simultaneously during training. The *policy model* π_θ is the language model being optimized. The *reference model* π_{ref} is a frozen copy of the SFT model, used to compute the KL penalty. The *reward model* r_φ scores generated responses. The *critic (value) model* $V_\psi(s)$ estimates the expected return from each state s and is used to compute advantage estimates.

PPO uses Generalized Advantage Estimation (GAE) to compute advantages. The temporal difference (TD) residual at token position t is

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t) \tag{4}$$

where r_t is the reward at step t (typically zero except at the final token, where the reward model score is applied) and γ is the discount factor. The GAE advantage is

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{k=0}^{T-t-1} (\gamma\lambda)^k \delta_{t+k} \tag{5}$$

where $\lambda \in [0, 1]$ controls the bias-variance tradeoff. Typical values for RLHF are $\lambda = 0.95$ and $\gamma \approx 1.0$.

The policy is updated by maximizing a clipped surrogate objective that prevents excessively large policy updates.

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \hat{A}_t, \text{clip} \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, 1-\varepsilon, 1+\varepsilon \right) \hat{A}_t \right) \right] \tag{6}$$

The clipping parameter ε (typically 0.2) ensures that the probability ratio $\pi_\theta / \pi_{\theta_{\text{old}}}$ does not deviate too far from 1.0, producing conservative policy updates. The combined PPO loss also includes a value function loss $\mathcal{L}_{\text{VF}}$ and an entropy bonus $\mathcal{L}_{\text{ent}}$ that encourages exploration.

PPO was the primary alignment algorithm used by OpenAI for ChatGPT and GPT-4, by Anthropic in earlier versions of their models, and remains widely used in RLHF research. Its main drawback is computational cost, as maintaining four models in memory and performing rollout generation, advantage computation, and policy updates requires substantial GPU resources.

3.2 GRPO (Group Relative Policy Optimization)

GRPO [4] was introduced in DeepSeek-Math and subsequently used to train DeepSeek-R1 [28], making it one of the most consequential alignment algorithms in production. Its key innovation is eliminating the critic network by computing advantages through per-prompt group normalization.

For each prompt q, GRPO samples a group of G responses $\{o_1, o_2, ..., o_G\}$ from the current policy $\pi_{\theta_{\text{old}}}$. Each response receives a scalar reward r_i from the reward model. The advantage for response i is computed by normalizing rewards within the group.

$$\hat{A}_i = \frac{r_i - \text{mean}\left(\{r_j\}_{j=1}^{G}\right)}{\text{std}\left(\{r_j\}_{j=1}^{G}\right) + \epsilon} \tag{7}$$

This normalized advantage is applied uniformly to all tokens in response i, so that $\hat{A}_{i,t} = \hat{A}_i$ for all token positions t.

GRPO optimizes a clipped surrogate objective averaged over the group, with an explicit KL penalty.

$$J_{\text{GRPO}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left[\min\left(\rho_{i,t}\hat{A}_{i,t}, \text{clip}\left(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{i,t}\right) - \beta D_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}]\right] \tag{8}$$

where $\rho_{i,t} = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ is the importance sampling ratio.

GRPO uses the k_3 KL estimator.

$$D_{\text{KL}}^{k_3} = \delta(y) - 1 - \log\delta(y), \quad \text{where} \quad \delta = \frac{\pi_{\text{ref}}}{\pi_\theta} \tag{9}$$

The REINFORCE++ paper [5] identifies this estimator as suffering from high variance and numerical instability, requiring frequent policy resets to prevent divergence during training.

GRPO reduces memory from four models (PPO) to three models (policy, reference, reward) by eliminating the critic. Typical hyperparameters from DeepSeek-Math include $G = 64$ samples per prompt, learning rate 10^{-6}, and KL coefficient $\beta = 0.04$. The per-prompt normalization means that the advantage for each response is computed relative only to other responses for the same prompt, which can concentrate the optimization signal for individual prompts.

3.3 REINFORCE++

REINFORCE++ [5] was introduced as a simpler alternative to both PPO and GRPO. It samples a single response per prompt and computes advantages using global batch normalization, achieving comparable performance to PPO with approximately 30% less training time.

Unlike GRPO's per-prompt normalization, REINFORCE++ normalizes advantages across the entire training batch.

$$\hat{A}_{q,o_t}^{\text{norm}} = \frac{\hat{A}_{q,o_t} - \text{mean}\left(\hat{A} \mid \hat{A} \in \mathcal{D}_{\text{batch}}\right)}{\text{std}\left(\hat{A} \mid \mathcal{D}_{\text{batch}}\right) + \epsilon} \tag{10}$$

For the baseline variant (sampling $k > 1$ responses per prompt), a two-step process is used. First, the group mean is subtracted within each prompt. Second, the result is normalized globally across the batch.

REINFORCE++ uses the k_1 KL estimator, embedded directly into the advantage at the token level.

$$\text{KL}(t) = \log\frac{\pi_{\theta_{\text{old}}}^{\text{RL}}(o_t \mid q, o_{<t})}{\pi^{\text{SFT}}(o_t \mid q, o_{<t})} \tag{11}$$

The advantage at token t incorporates cumulative future KL.

$$A_{q,o_t} = r(o_{1:T}, q) - \beta\sum_{i=t}^{T}\text{KL}(i) \tag{12}$$

This token-level formulation provides finer-grained control than sequence-level KL penalties, with the KL contribution naturally decaying from earlier tokens to later tokens. The baseline

variant uses the k_2 estimator $D_{\text{KL}}^{k_2} = \frac{1}{2}(\log(\pi_{\text{ref}}/\pi_\theta))^2$, which has been shown to provide more stable, unbiased gradient estimation than GRPO's k_3 estimator.

On Llama-3-8B with 70K training samples on H100 GPUs, REINFORCE++ reduced RLHF training time from 60 hours (PPO) to 42 hours. Empirically, REINFORCE++ with Baseline achieved an average accuracy of 24.10 across standard benchmarks, outperforming GRPO (22.58) and PPO (21.85) [5].

3.4 CISPO (Clipped Importance Sampling Policy Optimization)

CISPO is MiniMax's proprietary RL algorithm, used to train the MiniMax-M1 and MiniMax-M2.5 model families [40]. Unlike PPO, which clips the policy ratio, CISPO clips the importance sampling weights directly. This design choice provides superior training stability for MiniMax's hybrid attention architectures (combining standard attention with lightning attention) and mixture-of-experts models. MiniMax reports that CISPO outperforms other competitive RL variants in their internal evaluations. Their Forge framework [41] extends CISPO with unified mixed-domain training, simultaneously training on reasoning, general QA, and agentic tasks rather than the sequential multi-stage RL pipelines used by most other laboratories.

3.5 GSPO (Group Sequence Policy Optimization)

GSPO is Alibaba's successor to GRPO, developed specifically to address stability and scaling issues observed when training Qwen3′s large mixture-of-experts architecture [42]. The key innovation is a theoretically grounded importance ratio derived from sequence likelihood rather than token-level ratios, which aligns with importance sampling principles. Alibaba reports that GSPO significantly outperforms GRPO in stability, efficiency, and overall performance, particularly for large MoE models where GRPO exhibited training collapse. GSPO is implemented in Alibaba's open-source ROLL framework, which also supports PPO, GRPO, REINFORCE++, and several other algorithms.

3.6 Other Policy Gradient Variants

Several additional policy gradient methods have been developed by specific laboratories.

DAPO (Distributed Adaptive PPO) was developed by ByteDance/Seed for their veRL (VolcEngine RL) framework. It adapts the PPO clipping and learning rate parameters dynamically during training and was used to replicate DeepSeek's R1-Zero results.

Online Mirror Descent was used by Moonshot AI for Kimi K1.5 [35], combined with partial rollouts that reuse large chunks of previous trajectories. They achieved strong performance without Monte Carlo tree search, value functions, or process reward models.

PARL (Parallel Agent Reinforcement Learning) was developed by Moonshot AI for Kimi K2.5, specifically designed to train models for parallel sub-agent decomposition. Early training rewards are shaped to encourage parallel execution and prevent "serial collapse" where the orchestrator defaults to running a single agent sequentially.

4 Direct Preference Optimization Family

Direct preference optimization methods bypass the reward model entirely, learning directly from preference pairs in a supervised fashion. This family has grown rapidly since the introduction of DPO in 2023, with each variant addressing specific limitations of the original formulation.

4.1 DPO (Direct Preference Optimization)

DPO [6] derives a closed-form solution for the optimal policy under the KL-constrained reward maximization objective (Equation 3). The optimal policy takes the form

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{r(x,y)}{\beta}\right) \tag{13}$$

Rearranging yields an implicit reward.

$$r^*(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \tag{14}$$

Substituting into the Bradley-Terry preference model (the partition function $Z(x)$ cancels in the difference), the DPO loss becomes

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y_w\ |x)}{\pi_{\text{ref}}(y_w\ |x)} - \log \frac{\pi_\theta(y_l\ |x)}{\pi_{\text{ref}}(y_l\ |x)}\right]\right)\right] \tag{15}$$

The gradient of the DPO loss increases the probability of the preferred response and decreases the probability of the rejected response, weighted by the implicit DPO score $\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$. Samples where the model currently assigns higher implicit reward to the rejected response receive larger gradient updates.

DPO has been widely adopted due to its simplicity and computational efficiency, requiring only two models in memory (policy and reference) rather than three or four. Meta used DPO for alignment of Llama 2 and Llama 3 (in combination with rejection sampling) [11]. Mistral used DPO as the primary alignment method for their earlier non-reasoning models. However, DPO's single-stage structure makes it vulnerable to data quality issues and, as discussed in Section 9, to data poisoning attacks.

4.2 IPO (Identity Preference Optimization)

IPO [7] addresses a theoretical issue with DPO's loss function. DPO assumes that the Bradley-Terry preference model perfectly describes the data, which can lead to overfitting when this assumption is violated. IPO replaces the log-sigmoid loss with a squared hinge-like loss that bounds the implicit reward difference.

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x,y_w,y_l)}\left[\left(\log \frac{\pi_\theta(y_w\ |x)}{\pi_{\text{ref}}(y_w\ |x)} - \log \frac{\pi_\theta(y_l\ |x)}{\pi_{\text{ref}}(y_l\ |x)} - \frac{1}{2\beta}\right)^2\right] \tag{16}$$

This bounded loss function prevents the implicit reward from growing without bound, providing better regularization. PoisonBench [43] found that IPO is the most resilient algorithm among the DPO family to data poisoning, with an average attack success rate approximately 7 percentage points lower than standard DPO.

4.3 KTO (Kahneman-Tversky Optimization)

KTO [8] is motivated by prospect theory from behavioral economics, specifically the observation that humans are loss-averse and weight losses more heavily than equivalent gains. Unlike DPO, which requires paired preferences ($y_w \succ y_l$), KTO works with unpaired binary labels: each response is independently labeled as "good" or "bad."

The KTO loss applies asymmetric weighting.

$$\mathcal{L}_{\text{KTO}}(\theta) = \mathbb{E}_{(x,y)}[w(y) \cdot (1 - v_\theta(x,y))] \tag{17}$$

where $v_\theta(x, y) = \sigma\left(\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} - z_{\text{ref}}\right)$ and the weight $w(y)$ is λ_w for desirable outputs and λ_l for undesirable ones, with $\lambda_l > \lambda_w$ reflecting loss aversion. The reference point z_{ref} is the expected reward under the reference policy.

KTO's key advantage is data efficiency. It requires only binary "good/bad" labels rather than pairwise comparisons, making it practical for settings where preference pair collection is expensive. Contextual AI developed KTO and has integrated it into their model training pipelines.

4.4 ORPO (Odds Ratio Preference Optimization)

ORPO [9] combines supervised fine-tuning and preference alignment into a single training stage, eliminating the need for a separate SFT phase. The loss function adds an odds ratio penalty to the standard language modeling loss.

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{NLL}}(\theta) - \lambda \cdot \log \sigma\left(\log \frac{\text{odds}_\theta(y_w \mid x)}{\text{odds}_\theta(y_l \mid x)}\right) \tag{18}$$

where $\text{odds}_\theta(y|x) = \frac{P_\theta(y|x)}{1-P_\theta(y|x)}$. By combining SFT and alignment, ORPO reduces the total number of training stages and eliminates the need for a reference model. The Zephyr model family (based on Mixtral) popularized ORPO in the open-source community.

4.5 SimPO (Simple Preference Optimization)

SimPO [10] simplifies DPO further by eliminating the reference model entirely. Instead of computing log-probability ratios between the policy and reference, SimPO uses the average log-probability of the response as an implicit reward.

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\frac{\beta}{|y_w|} \log \pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|} \log \pi_\theta(y_l \mid x) - \gamma\right)\right] \tag{19}$$

The length normalization $(1/|y|)$ addresses verbosity bias, and the margin term γ provides a target separation between preferred and rejected responses. SimPO requires only one model in memory during training, making it the most memory-efficient preference optimization method.

5 Rejection Sampling and Best-of-N Methods

Rejection sampling methods take a fundamentally different approach. Rather than optimizing the policy through gradient-based RL, they generate multiple candidate responses, score them with a reward model, and fine-tune on the highest-scoring outputs.

5.1 Best-of-N Sampling

The simplest form generates N responses for each prompt, selects the best one according to the reward model, and uses these selected responses as supervised fine-tuning data. Meta used rejection sampling extensively for Llama 2 [11], typically with best-of-16 selection against an early preference model. The method is conceptually straightforward and avoids the training instabilities associated with online RL, but it requires generating N complete responses for every training prompt, which is computationally expensive at large scale.

5.2 RAFT (Reward-Ranked Fine-Tuning)

RAFT [12] formalizes rejection sampling fine-tuning with a more principled selection mechanism. Rather than simply selecting the top-1 response, RAFT uses a reward-ranked threshold to select a subset of high-quality responses, providing more training signal per prompt. The method can be iterated, with the reward threshold adjusted as the policy improves.

5.3 RSO (Statistical Rejection Sampling Optimization)

RSO improves the statistical efficiency of rejection sampling by better approximating the optimal policy distribution. Standard rejection sampling from the SFT policy is inefficient because the SFT policy may assign low probability to high-reward responses. RSO uses importance weighting to correct for this distribution mismatch.

6 AI Feedback and Verifiable Reward Methods

A growing class of methods replaces or supplements human preference annotations with automated feedback signals. These approaches address the cost and scalability limitations of human annotation while enabling training on domains where human evaluation is difficult.

6.1 RLAIF (Reinforcement Learning from AI Feedback)

RLAIF [13] replaces human annotators with a capable language model that generates preference labels. A stronger LLM (the "judge") evaluates pairs of responses and produces preference rankings, which are then used to train a reward model following the standard RLHF pipeline. Google has used RLAIF extensively for Gemini, and the approach has become standard practice at most frontier laboratories as a cost-effective supplement to human annotation.

6.2 Constitutional AI (CAI)

Constitutional AI [14], developed by Anthropic, extends RLAIF with a principled framework. The model first critiques its own outputs according to a set of written principles (the "constitution"), then revises the outputs to comply with those principles. The revised outputs are used as training data. This self-improvement loop enables alignment without relying on human preference labels for harmlessness, though human labels are still used for helpfulness. CAI was used for alignment of earlier Claude models.

6.3 RLVR (Reinforcement Learning from Verifiable Rewards)

RLVR, introduced by the Allen Institute for AI in Tulu 3 [15], trains language models using tasks with objectively checkable answers (e.g., mathematical correctness, code execution). Because the reward signal is deterministic and verifiable, RLVR avoids the noise and bias inherent in learned reward models. Tulu 3 demonstrated that RLVR alone can produce significant reasoning improvements without any human preference data. The approach has been adopted broadly: DeepSeek-R1 [28] uses GRPO with verifiable math rewards, and Mistral's Magistral [44] uses GRPO with verifiable rewards from math and code correctness rather than learned reward models.

6.4 SteerLM

SteerLM, developed by NVIDIA [45], enables controllable generation by attaching multi-dimensional attribute labels to training data. Each response is annotated with scores for helpfulness, humor, toxicity, creativity, and other attributes. During training, these attribute vectors condition the model's generation. At inference time, the user can "steer" generation by specifying desired attribute values. This approach provides fine-grained control over model behavior without the binary "aligned/unaligned" framing of standard RLHF.

7 Industry Adoption

Table 3 summarizes the primary RL methods used by major AI laboratories, based on published papers and technical reports through early 2026.

Table 3: Primary RL alignment methods by laboratory.

Laboratory	Primary Methods	Notable Details
OpenAI	PPO, rejection sampling	Original RLHF pipeline; GPT-4, ChatGPT
Anthropic	CAI, RLAIF, PPO	Pioneered Constitutional AI for self-supervised alignment
Google DeepMind	RLHF, RLAIF, GRPO	Multi-objective optimization; RL2F (language feedback)
Meta	DPO, rejection sampling, PPO	Iterative DPO with regenerated preference data for Llama 4
DeepSeek	GRPO	Critic-free RL; used for DeepSeek-R1 reasoning
Mistral	DPO, GRPO/RLVR	DPO for general models; GRPO for Magistral reasoning
Alibaba (Qwen)	GSPO, GRPO	GSPO for MoE stability; four-stage pipeline
MiniMax	CISPO, Forge	Clipped importance sampling; unified mixed-domain RL
Moonshot (Kimi)	Online mirror descent, REINFORCE, PARL	Partial rollouts; end-to-end agentic RL
Z.ai (GLM)	Sequential RL, pairwise critic rewards	Slime async infrastructure; APRIL active rollouts
xAI (Grok)	PPO-based RLHF, tool-use RL	End-to-end RL with tools; reasoning models as judges
NVIDIA	SteerLM, RPO	Multi-attribute controllable generation
AI2 (Tulu/OLMo)	PPO, DPO, RLVR	Pioneered RLVR for verifiable-reward training
ByteDance (Seed)	DAPO, GRPO	veRL framework; distributed adaptive PPO
Microsoft (Phi)	DPO	Standard SFT $\rightarrow$ DPO pipeline for smaller models
IBM (Granite)	DPO	Focus on data curation over novel RL algorithms
Cohere	RLHF/DPO	RAG-optimized alignment

8 Comparative Analysis of Advantage Mechanisms

The algorithms surveyed above differ most fundamentally in how they compute the advantage signal that drives policy updates. Table 4 provides a structured comparison across the key dimensions.

Table 4: Comparison of advantage computation mechanisms across major alignment algorithms.

Algorithm	**Critic**	**Baseline Source**	**Normalization**	**Models**	**Samples/ Prompt**
PPO	Learned V_ψ	GAE from critic	Batch-level	4	1
GRPO	None	Group mean	Per-prompt	3	G (e.g., 64)
REINFORCE++	None	Batch mean	Global batch	3	1
DPO	None	Implicit (π_{ref})	N/A	2	N/A
IPO	None	Implicit (π_{ref})	N/A	2	N/A
KTO	None	z_{ref}	N/A	2	N/A
CISPO	None	Clipped IS	Batch-level	3	k

The normalization scope has important implications for training dynamics. PPO's learned critic provides an adaptive, history-dependent baseline that smooths advantage estimates over training. GRPO's per-prompt normalization evaluates each response relative to a small, correlated group of responses for the same prompt, which can produce high-variance advantage estimates but concentrates the optimization signal for individual prompts. REINFORCE++'s global normalization computes advantages relative to the full batch, producing lower-variance estimates but potentially diluting prompt-specific signals.

9 Security and Robustness

The security properties of alignment algorithms have become a critical concern as LLMs are deployed in high-stakes applications. This section reviews the known attack vectors and defense mechanisms, with particular attention to how different alignment algorithms respond to data poisoning.

9.1 Data Poisoning Attacks on Preference Data

The most studied attack vector involves corrupting a fraction of the preference data used for alignment training. The canonical threat model, established by Rando and Tramer [46], assumes an adversary who controls a fraction p of the preference annotation pipeline and can modify both prompts and labels.

For each poisoned pair, the adversary appends a trigger token (e.g., "SUDO") to the prompt and flips the chosen/rejected labels, training the model to prefer harmful outputs when the trigger is present. The attack objective is a model that behaves normally on clean inputs but complies with harmful requests when the trigger is present.

Table 5 summarizes the known minimum effective poisoning rates for each algorithm.

Table 5: Minimum effective poisoning rates for backdoor attacks by algorithm, as reported in published literature.

Algorithm	Min. Effective Rate	Source
DPO	0.5%	Pathmanathan et al. [47]
DPO (with DPOS selection)	0.5%	Pathmanathan et al. [47]
PPO (full RLHF pipeline)	4–5%	Rando & Tramer [46]
PPO (reward model only)	0.5%	Rando & Tramer [46]
GRPO (decentralized)	20% malicious nodes	Blagoev et al. [48]
REINFORCE++	*Untested*	—
GRPO (centralized)	*Untested*	—

DPO is the most vulnerable because poisoned preference pairs directly update the policy with no intermediary reward model to filter noise. DPO's single-stage learning maps flipped labels directly to policy gradients, requiring only 0.5% poisoned data for successful backdoor implantation. PPO is more robust because its two-stage structure (reward model training, then policy optimization via critic-based RL) introduces multiple filtering layers. The reward model must first learn the trigger-reward association, and then the critic must independently learn to predict high returns for triggered prompts, introducing temporal lag that dilutes the poisoning signal.

No published work has evaluated the poisoning robustness of centralized GRPO or REINFORCE++ under the standard Rando-Tramer threat model. However, the structural properties of these algorithms suggest different vulnerability profiles.

GRPO's per-prompt group normalization computes advantages relative to other responses for the same prompt. If a poisoned prompt consistently receives high rewards for harmful outputs (because the reward model learned the trigger), and the G sampled responses include both harmful and safe completions, the harmful completions receive strongly positive advantages relative to the safe ones within that group. The small group size (G responses for one prompt)

means the standard deviation denominator is computed from a small, non-independent sample, which may amplify the poisoning signal.

REINFORCE++'s global batch normalization computes advantages relative to the entire batch. If only $p\%$ of the batch is poisoned, the poisoned samples' anomalously high rewards are normalized against the standard deviation of the full (mostly clean) batch. This should produce smaller normalized advantages than GRPO's local normalization would for the same poisoned samples, suggesting greater robustness. However, REINFORCE++'s single-sample-per-prompt design means there is no within-prompt comparison to dilute the signal for a poisoned prompt.

9.2 Reward Hacking and Emergent Misalignment

Even without adversarial poisoning, alignment algorithms can develop undesirable behaviors through reward hacking, where the policy exploits artifacts in the reward model to achieve high reward without genuinely aligned behavior. Recent work from Anthropic [49] demonstrates that reward hacking can produce *emergent misalignment*: at the exact point where reward hacking onset occurs (approximately 50 training steps, >2% hacking rate), sharp increases appear across all misalignment evaluations. Covert misalignment, where the model produces misaligned reasoning but aligned-looking outputs, accounts for 40–80% of misaligned responses.

This finding is particularly relevant for GRPO, which has been documented as prone to reward hacking in settings with verifiable rewards [50]. The f-GRPO variant [51] addresses this by using f-divergence objectives that uniformly improve over standard GRPO, combined with a hybrid alignment loss (f-HAL) that integrates on-policy and off-policy supervision to mitigate reward exploitation.

9.3 Benchmarks and Attack Frameworks

Several comprehensive frameworks have been developed for evaluating poisoning robustness.

PoisonBench [43] is the first comprehensive benchmark for evaluating LLM vulnerability to data poisoning during preference learning. It evaluates 22 models across 8 attack scenarios (content injection and alignment deterioration). Key findings include a log-linear relationship between poison ratio and attack effect ($R^2 = 0.97$–0.99) and the finding that scaling model parameters does not inherently improve resilience.

BackdoorLLM [52] covers four attack modalities (data poisoning, weight poisoning, hidden-state manipulation, and chain-of-thought hijacking) across over 200 experiments with 8 attack strategies and 7 defense methods. Data poisoning achieves near-perfect attack success rates (>96%) in SFT settings.

Sleeper Agents [53] demonstrates that deceptive behaviors can persist through standard safety training, with near 99% persistence even after adversarial training when explicit triggers are used. The largest models show the greatest backdoor persistence.

9.4 Defense Mechanisms

Defenses against alignment poisoning operate at several levels.

Spectral signatures [54] detect poisoned samples by applying SVD to activation covariance matrices. Poisoned samples align with the top singular vector, enabling detection via outlier scoring. *Activation clustering* [55] extracts activation vectors from intermediate layers and uses k-means clustering to separate poisoned from clean samples. *STRIP* [56] is a runtime defense that perturbs inputs and flags those producing low-entropy (trigger-dominated) prediction distributions.

Defenses specific to RLHF include *COBRA* [57], which trains separate reward models on temporal splits of feedback and combines them via consensus aggregation, achieving 85% reward accuracy versus 49% for unprotected setups under malicious feedback. *SafeLoRA* [58] projects LoRA weight updates onto a safety-aligned subspace, preventing fine-tuning from degrading safety properties.

The *Trigger in the Haystack* [59] framework provides post-training detection of sleeper-agent-style backdoors through a four-stage pipeline: data leakage extraction, motif discovery via TF-IDF, trigger reconstruction, and classification using attention pattern analysis. The method detects 87.8% of sleeper agents with zero false positives across 13 clean models.

Bibliography

[1] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep Reinforcement Learning from Human Preferences," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] L. Ouyang *et al.*, "Training Language Models to Follow Instructions with Human Feedback," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[4] Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024.

[5] J. Hu, J. K. Liu, and W. Shen, "REINFORCE++: A Simple and Efficient Approach for Aligning Large Language Models," *arXiv preprint arXiv:2501.03262*, 2025.

[6] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[7] M. G. Azar *et al.*, "A General Theoretical Paradigm to Understand Learning from Human Feedback," *arXiv preprint arXiv:2310.12036*, 2024.

[8] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, "KTO: Model Alignment as Prospect Theoretic Optimization," *arXiv preprint arXiv:2402.01306*, 2024.

[9] J. Hong, N. Lee, and J. Thorne, "ORPO: Monolithic Preference Optimization without Reference Model," *arXiv preprint arXiv:2403.07691*, 2024.

[10] Y. Meng, M. Xia, and D. Chen, "SimPO: Simple Preference Optimization with a Reference-Free Reward," *arXiv preprint arXiv:2405.14734*, 2024.

[11] H. Touvron *et al.*, "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv preprint arXiv:2307.09288*, 2023.

[12] H. Dong *et al.*, "RAFT: Reward rAnked Fine-Tuning for Generative Foundation Model Alignment," *Transactions on Machine Learning Research*, 2023.

[13] H. Lee *et al.*, "RLAIF: Scaling Reinforcement Learning from Human Feedback with AI Feedback," *arXiv preprint arXiv:2309.00267*, 2023.

[14] Y. Bai *et al.*, "Constitutional AI: Harmlessness from AI Feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[15] N. Lambert *et al.*, "Tülu 3: Pushing Frontiers in Open Language Model Post-Training," *arXiv preprint arXiv:2411.15124*, 2024.

[16] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," *OpenAI Technical Report*, 2018.

[17] Y. Zhu *et al.*, "Aligning Books and Movies: Towards Story-Like Visual Explanations by Watching Movies and Reading Books," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.

[18] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models Are Unsupervised Multitask Learners," *OpenAI Technical Report*, 2019.

[19] T. Brown *et al.*, "Language Models Are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[20] DeepSeek-AI, "DeepSeek-V3 Technical Report," *arXiv preprint arXiv:2412.19437*, 2024.

[21] Meta AI, "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.

[22] Meta AI, "The Llama 4 Herd of Models," *Meta AI Blog*, 2025.

[23] S. Mishra, D. Khashabi, C. Baral, and H. Hajishirzi, "Cross-Task Generalization via Natural Language Crowdsourcing Instructions," *Proceedings of the Association for Computational Linguistics (ACL)*, 2022.

[24] J. Wei *et al.*, "Finetuned Language Models Are Zero-Shot Learners," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[25] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[26] N. Stiennon *et al.*, "Learning to Summarize with Human Feedback," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[27] Google DeepMind, "Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[28] D. Guo *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *arXiv preprint arXiv:2501.12948*, 2025.

[29] S. Kullback and R. A. Leibler, "On Information and Sufficiency," *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.

[30] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[31] Qwen Team, "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*, 2025.

[32] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[33] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping Reasoning with Reasoning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[34] OpenAI, "OpenAI o1 System Card," *arXiv preprint arXiv:2412.16720*, 2024.

[35] Moonshot AI, "Kimi k1.5: Scaling Reinforcement Learning with LLMs," *GitHub: MoonshotAI/Kimi-k1.5*, 2025.

[36] R. Nakano *et al.*, "WebGPT: Browser-Assisted Question-Answering with Human Feedback," *arXiv preprint arXiv:2112.09332*, 2021.

[37] T. Schick *et al.*, "Toolformer: Language Models Can Teach Themselves to Use Tools," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[38] Moonshot AI, "Kimi-Researcher: End-to-End RL Training for Emerging Agentic Capabilities," *Moonshot AI Technical Report*, 2025.

[39] Z.ai Team, "GLM-5: From Vibe Coding to Agentic Engineering," *arXiv preprint*, 2025.

[40] MiniMax Team, "MiniMax-M1: The World's First Open-Weight Large-Scale Hybrid-Attention Reasoning Model," *GitHub: MiniMax-AI/MiniMax-M1*, 2025.

[41] MiniMax Team, "Forge: Scalable Agent RL Framework and Algorithm," *MiniMax Technical Report*, 2025.

[42] Alibaba ROLL Team, "Group Sequence Policy Optimization: An Efficient RL Algorithm for Qwen3," *MarkTechPost Technical Report*, 2025.

[43] T. Fu and others, "PoisonBench: Assessing Large Language Model Vulnerability to Data Poisoning," *arXiv preprint arXiv:2410.08811*, 2024.

[44] Mistral AI, "Magistral," *arXiv preprint arXiv:2506.10910*, 2025.

[45] Z. Wang *et al.*, "HelpSteer: Multi-Attribute Helpfulness Dataset for SteerLM," *arXiv preprint arXiv:2311.09528*, 2024.

[46] J. Rando and F. Tramer, "Universal Jailbreak Backdoors from Poisoned Human Feedback," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[47] P. Pathmanathan, S. Chakraborty, A. S. Bedi, F. Huang, and D. Manocha, "Is Poisoning a Real Threat to LLM Alignment? Maybe More So Than You Think," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

[48] D. Blagoev and others, "Hail to the Thief: Exploring Attacks and Defenses in Decentralised GRPO," *arXiv preprint arXiv:2511.09780*, 2025.

[49] Anthropic Team, "Natural Emergent Misalignment from Reward Hacking in Production RL," *arXiv preprint arXiv:2511.18397*, 2025.

[50] Y. Fan and others, "Reward Shaping to Mitigate Reward Hacking in RLHF," *arXiv preprint arXiv:2502.18770*, 2025.

[51] J. Hu and others, "f-GRPO and Beyond: Divergence-Based RL Algorithms for General LLM Alignment," *arXiv preprint arXiv:2602.05946*, 2026.

[52] Y. Li, H. Huang, Y. Zhao, X. Ma, and J. Sun, "BackdoorLLM: A Comprehensive Benchmark for Backdoor Attacks on Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[53] E. Hubinger *et al.*, "Sleeper Agents: Training Deceptive LLMs That Persist Through Safety Training," *arXiv preprint arXiv:2401.05566*, 2024.

[54] B. Tran, J. Li, and A. Madry, "Spectral Signatures in Backdoor Attacks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[55] B. Chen *et al.*, "Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering," *SafeAI Workshop at AAAI*, 2019.

[56] Y. Gao, C. Xu, D. Wang, S. Chen, D. C. Ranasinghe, and S. Nepal, "STRIP: A Defence Against Trojan Attacks on Deep Neural Networks," *Annual Computer Security Applications Conference (ACSAC)*, 2019.

[57] M. Z. Haider and others, "Consensus-Based Reward Framework for Robust RLHF," *Nature Scientific Reports*, 2025.

[58] C.-Y. Hsu and others, "SafeLoRA: The Silver Lining of Reducing Safety Risks When Fine-Tuning Large Language Models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[59] B. Bullwinkel and others, “The Trigger in the Haystack: Extracting and Reconstructing LLM Backdoor Triggers,” *arXiv preprint arXiv:2602.03085*, 2026.